

PROGRAMMING FOR PROBLEM SOLVING (PPS)

UNIT-1

Logic Building

Flow chart, Algorithm, Pseudo code.

Introduction to Programming

Computer Languages, Creating and running programs, Program Development.

Introduction to the C Language

Background, C Programs, Identifiers, Data Types, Variables, Constants, Input/output functions.

Operators

Arithmetic, relational, logical, bitwise, conditional, increment/decrement, assignment. Expressions, Precedence and Associativity, Expression Evaluation, Type conversions.

Logic Building:

Flow chart, Algorithm, Pseudo code.

Algorithm: *Algorithm* is a finite set of instructions that, if followed accomplishes a particular task. Algorithm should satisfy the following criteria

- **Input:** Zero or more quantities are externally supplied.
- **Output:** At least one quantity is produced.
- **Definiteness:** Each instruction is clear and unambiguous. Ex: Add B or C to A
- **Finiteness:** Algorithm should terminate after finite number of steps when traced in all cases. Ex: Go on adding elements to an array
- **Effectiveness:** Every instruction must be basic i.e., it can be carried out, by a person using pencil and paper.

Algorithm must also be general to deal with any situation.

Advantages of Algorithms:

- It provides the core solution to a given problem. the solution can be implemented on a computer system using any programming language of user's choice.
- It facilitates program development by acting as a design document or a blue print of a given problem solution.
- It ensures easy comprehension of a problem solution as compared to an equivalent computer program.

- It eases identification and removal of logical errors in a program.
- It facilitates algorithm analysis to find out the most efficient solution to a given problem.

Disadvantages of Algorithms:

- In large algorithms the flow of program control becomes difficult to track.
- Algorithms lack visual representation of programming constructs like flowcharts;
- thus, understanding the logic becomes relatively difficult.

The same problem can be solved with different methods. So, to solve a problem different algorithm, may be accomplished. Algorithm may vary in time, space utilized. User writes algorithm in his / her own language. So, it cannot be executed on computer. Algorithm should be in sufficient detail that it can be easily translated into any of the languages.

Example1: Add two numbers.

Step 1: Start

Step 2: Read 2 numbers as A and B

Step 3: Add numbers A and B and store result in C

Step 4: Display C

Step 5: Stop

Example2: Average of 3 numbers.

Step 1: Start

Step 2: Read the numbers a, b, c

Step 3: Compute the sum and divide by 3

Step 4: Store the result in variable d

Step 5: Print value of d

Step 6: End

Example3: Average of n inputted numbers.

Step 1: Start

Step 2: Read the number n

Step 3: Initialize i to zero

Step 4: Initialize sum to zero

Step 5: If i is greater than n

Step 6: Read a

Step 7: Add a to sum

Step 8: Go to step 5

Step 9: Divide sum by n & store the result in avg

Step 10: Print value of avg

Step 11: End

Flowchart:

A *flowchart* is a visual representation of the sequence of steps for solving a problem. A *flowchart* is a set of symbols that indicate various operations in the *program*. For every process, there is a corresponding *symbol* in the *flowchart*. Once an *algorithm* is written, its pictorial representation can be done using *flowchart symbols*. In other words, a pictorial representation of a *textual algorithm* is done using a *flowchart*. A *flowchart* gives a pictorial representation of an *algorithm*.

- The first *flowchart* is made by John Von Neumann in 1945.
- It is a symbolic diagram of operations sequence, dataflow, control flow and processing logic in information processing.
- The symbols used are simple and easy to learn.
- It is a very helpful tool for programmers and beginners.

Purpose of a Flowchart:

- Provides communication.
- Provides an overview.
- Shows all elements and their relationships.
- Quick method of showing program flow.
- Checks program logic.
- Facilitates coding.
- Provides program revision.
- Provides program documentation.

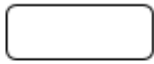
Advantages of a Flowchart:

- Flowchart is an important aid in the development of an algorithm itself.
- Easier to understand than a program itself.
- Independent of any particular programming language.
- Proper documentation.
- Proper debugging.
- Easy and clear presentation.

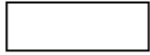
Limitations of a Flowchart:

- Complex logic.
- Drawing is time consuming.
- Difficult to draw and remember.
- Technical detail.

Flowchart Symbols



Terminal symbol - indicates the beginning and end points of an algorithm.



Process symbol - shows an instruction other than input, output or selection.



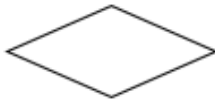
Input-output symbol - shows an input or an output operation.



Disk storage I/O symbol - indicates input from or output to disk storage.



Printer output symbol - shows hardcopy printer output.



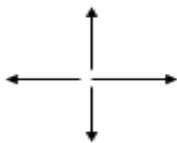
Selection symbol - shows a selection process for two-way selection.



Off-page connector - provides continuation of a logical path on another page.



On-page connector - provides continuation of logical path at another point in the same page.



Flow lines - indicate the logical sequence of execution steps in the algorithm.

Flowchart – Example

1) algorithm and flow chart for swapping two numbers

To Swap two integer numbers:

Algorithm: using third variable

Step 1: Start

Step 2: Input num1, num2

Step 3: temp = num1

Step 4: $\text{num1} = \text{num2}$

Step 5: $\text{num2} = \text{temp}$

Step 6: Output num1, num2

Step 7: Stop

Algorithm: without using third variable

Step 1: Start

Step 2: Input num1, num2

Step 3: calculate $\text{num1} = \text{num1} + \text{num2}$

Step 4: calculate $\text{num2} = \text{num1} - \text{num2}$

Step 5: calculate $\text{num1} = \text{num1} - \text{num2}$

Step 6: Output num1, num2

Step 7: Stop

Flowcharts:

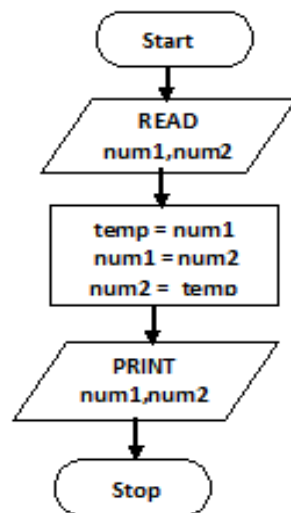


Fig a: With using third variable

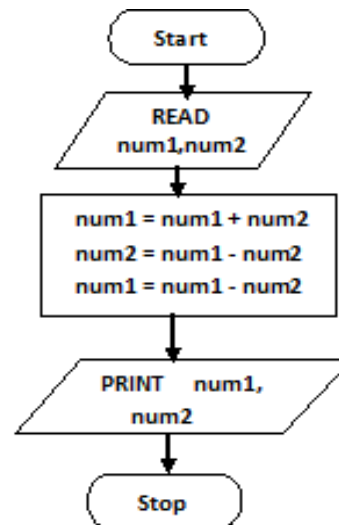


Fig b: Without using third variable

2. algorithm and flowchart to find the largest among two numbers.

Algorithm:

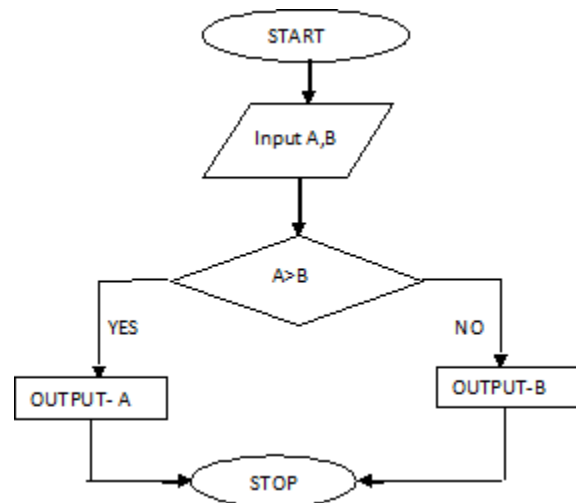
Step 1: Start

Step 2: Input A , B

Step 3: if $A > B$ then output A else output B

Step 4: Stop

Flowchart



3. algorithm and flowchart to find the largest among three numbers.

Algorithm:

Step 1: Start

Step 2: Input A, B, C

Step 3: if $A > B$ go to step 4 , otherwise go to step 5

Step 4: if $A > C$ go to step 6 , otherwise go to step 8

Step 5: if $B > C$ go to step 7, otherwise go to step 8

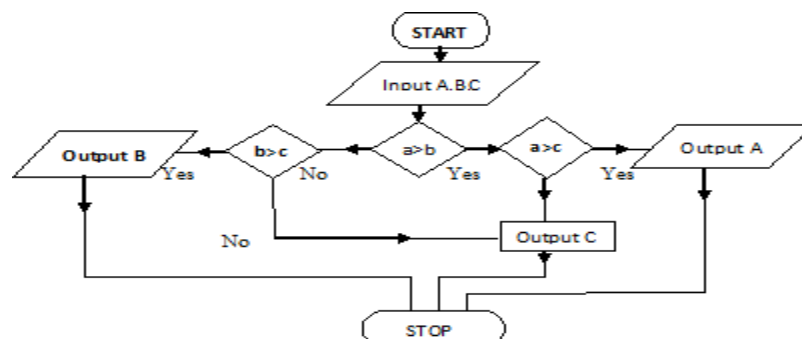
Step 6: print — A is largest | , go to step 9

Step 7: print — B is largest | , go to step 9

Step 8: print — C is largest | , go to step 9

Step 9: Stop

Flowchart:



4. algorithm and flowchart to find factorial of a number [Hint: fact (4) = 1 * 2* 3* 4]

Algorithm:

Step 1: Start

Step 2: Input n

Step 3: Initialize counter variable, i , to 1 and factors = 1

Step 4: if $i \leq n$ go to step 5 otherwise go to step 7

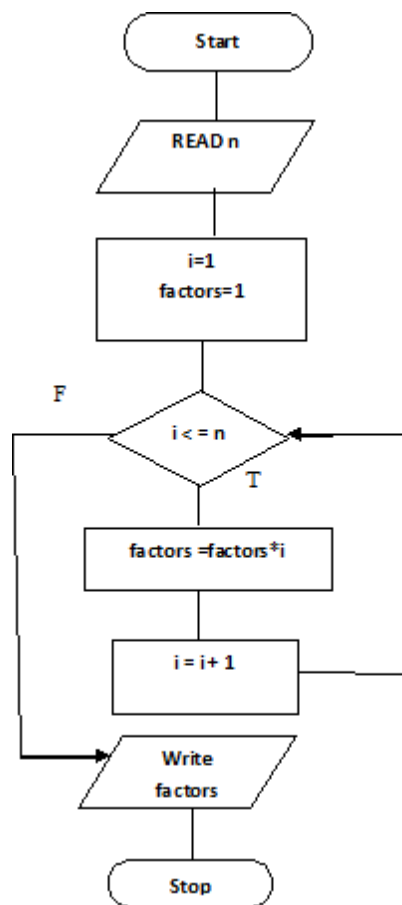
Step 5: calculate $\text{factors} = \text{factors} * i$

Step 6: increment counter variable, i, and go to step 4

Step 7: output factors.

Step 8: stop

Flowchart:



5. algorithm and flowchart to find whether a number is prime or not. [Hint: A number is said to be prime number for which the only factors are 1 and itself]

Algorithm:

Step 1: Start

Step 2: Input n

Step 3: Let $i = 1$, count=0

Step 4: if $i > n/2$ go to step 7

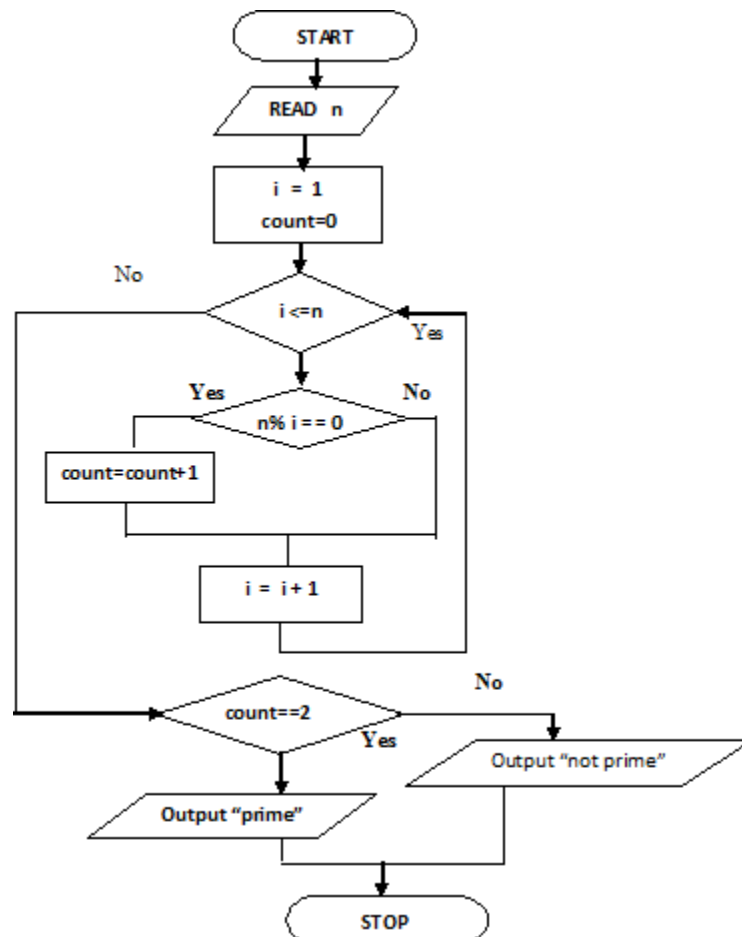
Step 5: if $(n \% i == 0)$ count = count + 1

Step 6: increment i and go to step 4

Step 7: if count=2 then print "prime number" else print "not prime number"

Step 8: Stop

Flow chart:



6. Write an algorithm and flowchart to find reverse of a number.

Algorithm:

Step 1: Start

Step 2: Input n

Step 3: Let rev=0

Step 4: if n<=0 go to step 8

Step 5: digit = n % 10

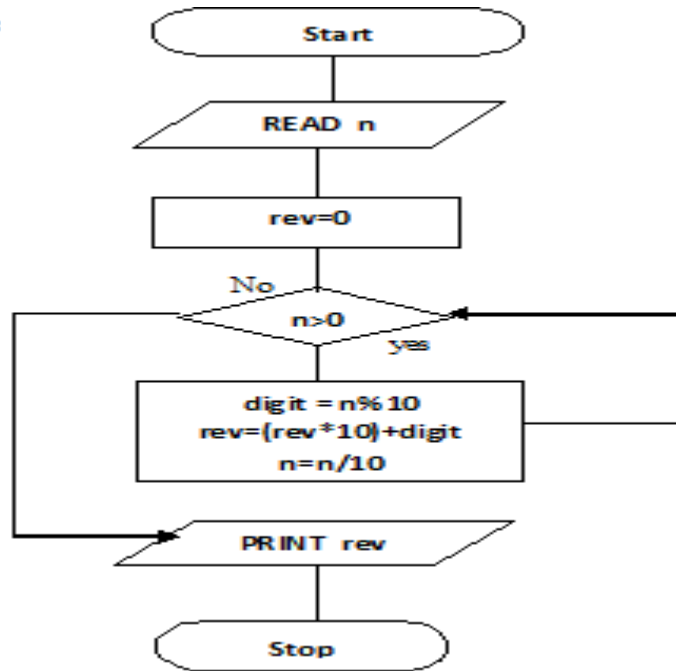
Step 6: rev = (rev * 10) + digit

Step 7: n = n / 10 then go to step 4

Step 8: output rev

Step 9: Stop

Flow chart:



7. Write an algorithm and flowchart to find the Sum of individual digits if a given number

Algorithm: -

Step 1 : Start

Step 2 : read n

Step 3 : Sum = 0

Step 4 : While n > 0 do

Step 4.1 : r = n % 10

Step 4.2 : Sum = Sum + r

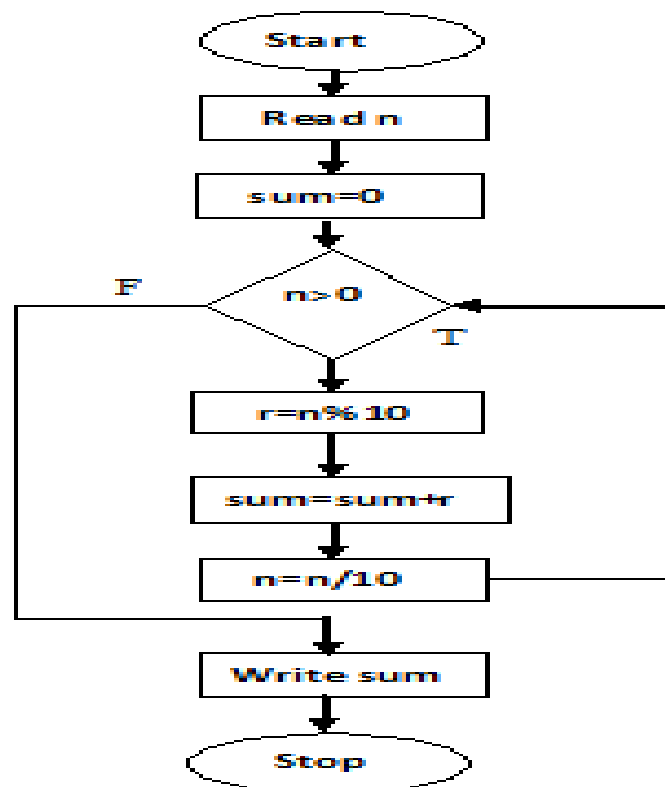
Step 4.3 : n = n/10

Step 5 : End while

Step 6 : Write Sum

Step 7 : Stop

Flowchart: -



Pseudo code:

- Pseudocode is one of the methods that could be used to represent an algorithm.
- It is not written in a specific syntax, that is used by a programming language and therefore cannot be executed in a computer.
- There are lots of formats used for writing pseudocodes
- Also, natural language is used in Pseudocode when presenting details that are not important.
- Most of the algorithms are presented using pseudocode, since they can be read and understood using programmers, who are familiar with different programming languages.
- Some programming constructs used for Pseudo Code
 - READ, PRINT, SET, INITIALISE, INCREMENT, IF - THEN – ENDIF, IF – THEN - ELSE - ENDIF, REPEAT - UNTIL, DO - WHILE etc.
- It is convenient and user friendly.

Algorithm Vs Pseudocode

- An algorithm is a well-defined sequence of steps that provides a solution for a given problem, while a pseudocode is one of the methods that can be used to represent an algorithm.
- While algorithms can be written in natural language, pseudocode is written in a format that is closely related to high level programming language structures.
- But pseudocode does not use specific programming language syntax and therefore could be understood by programmers who are familiar with different programming languages.

- Additionally, transforming an algorithm presented in pseudocode to programming code could be much easier than converting an algorithm written in natural language.

Problem-1: Write a program to find the average of n numbers.

Algorithm:

1. Read the value of n
2. Read n numbers
3. Add these n numbers
4. Divide the addition result by n to get the average
5. Print the average

Pseudo Code:

1. Start
2. Read n
3. sum ← 0
4. for i 1 to n do
5. read i-th number
6. sum ← sum + i-th number
7. end for
8. average ← sum/n
9. print average
10. stop

Introduction to Programming:

Computer Languages, Creating and running programs, Program Development.

Computer Languages:

- Computer language is defined as code or syntax which is used to write programs or any specific applications
- The computer language is used to communicate with computers
- Three categories assembly language, machine language, and high-level language

Types of Computer Language

1. Machine languages (low level languages)
2. Assembly (or symbolic) languages

3. Procedure-oriented languages (high level languages)

1. Machine language:

Computers are made of two-state electronic devices they can understand only pulse and no-pulse (or `_1_` and `_0_`) conditions. Therefore, all instructions and data should be written using binary codes 1 and 0. This binary code is called the machine code or machine language. Computers do not understand English, Hindi or Telugu. They respond only to machine language. Added to this, computers are not identical in design, therefore, each computer has its own machine language. (However, the script 1 and 0, is the same for all computers). This poses two problems for the user.

- It is difficult to understand and remember the various combinations of 1_s and 0_s representing numerous data and instructions. Also, writing error-free instructions is a slow process.
- As every machine has its own machine language, the user cannot communicate with other computers. Machine languages are usually referred to as the first-generation languages.

2. Assembly language:

The Assembly language which is introduced in 1950s, reduced programming complexity and provided some standardization to build an application. The assembly language, also referred to as the second-generation programming language, is also a low-level language. In an assembly language, the 0s and 1s of machine language are replaced with abbreviations or mnemonic code.

The main advantages of an assembly language over a machine language are:

- As we can locate and identify syntax errors in assembly language, it is easy to debug it.
- It is easier to develop a computer application using assembly language in comparison to machine language.
- Assembly language operates very efficiently.

An assembly language program consists of a series of instructions and mnemonics that correspond to a stream of executable instructions. An assembly language instruction consists of a mnemonic code followed by zero or more operands. The mnemonic code is called the operation code or opcode, which specifies the operation to be performed on the given arguments.

During the execution, the assembly language program is converted into the machine code with the help of an assembler. The simple assembly language statements had one-to-one correspondence with the machine language statements. This one-to-one correspondence still generated complex programs. Then, macroinstructions were devised so that multiple machine language statements could be represented using a single assembly language instruction. Even today programmers prefer to use an assembly language for performing certain tasks such as:

- To initialize and test the system hardware prior to booting the operating system. This assembly language code is stored in ROM.
- To write patches for disassembling viruses, in anti-virus product development companies.

- To attain extreme optimization, for example, in an inner loop in a processor- intensive algorithm.
- For direct interaction with the hardware.
- In extremely high-security situations where complete control over the environment is required.
- To maximize the use of limited resources, in a system with serve resource constraints.

3. High-level languages:

High level languages further simplified programming tasks by reducing the number of computer operation details that had to be specified. High level languages like COBOL, Pascal, FORTRAN, and C are more abstract, easier to use, and more portable across platforms, as compared to low level programming languages. Instead of dealing with registers, memory addresses and call stacks, a programmer can concentrate more on the logic to solve the problem with help of variables, arrays or Boolean expressions.

High-level languages can be classified into the following three categories:

- Procedure-oriented languages (third generation)
- Problem-oriented languages (fourth generation)
- Natural languages (fifth generation)

Procedure-oriented languages

High level languages designed to solve general-purpose problems are called procedural languages or third generation languages. These include BASIC, COBOL, FORTRAN, C, C++, and JAVA, which are designed to express the logic and procedure of a problem.

Problem-oriented languages

Problem-oriented languages are used to solve specific problems and are known as the fourth-generation languages. These include query languages, report generators and application generators which have simple, English-like syntax rules.

Natural languages

Natural languages are designed to make a computer to behave like an expert and solve problems. The programmer just needs to specify the problem and the constraints for problem-solving. Natural languages such as LISP and PROLOG are mainly used to develop artificial intelligence and expert systems. These languages are widely known as fifth generation languages.

Creating and running programs:

It is the job of programmer to write and test the program. The following are four steps for creating and running programs:

- Writing and Editing the Program.
- Compiling the Program.
- Linking the Program with the required library modules.
- Executing the Program.

A. Writing and Editing Program:

The Software to write programs is known as text editor. A text editor helps us enter, change and store character data. Depending on the editor on our system, it could be used to write letters, create reports or write programs.

Example: word processor.

The text editor could be generalized word processor, but every compiler comes with associated text editor.

Some of the features of editors are

- Search: To locate and replace statements.
- Copy, Paste: To copy and move statements.
- Format: To set tabs to align text.

After the program is completed the program is saved in a file to disk. This file will be input to the compiler, it is known as source file. The following figure shows the various steps in building a C – program

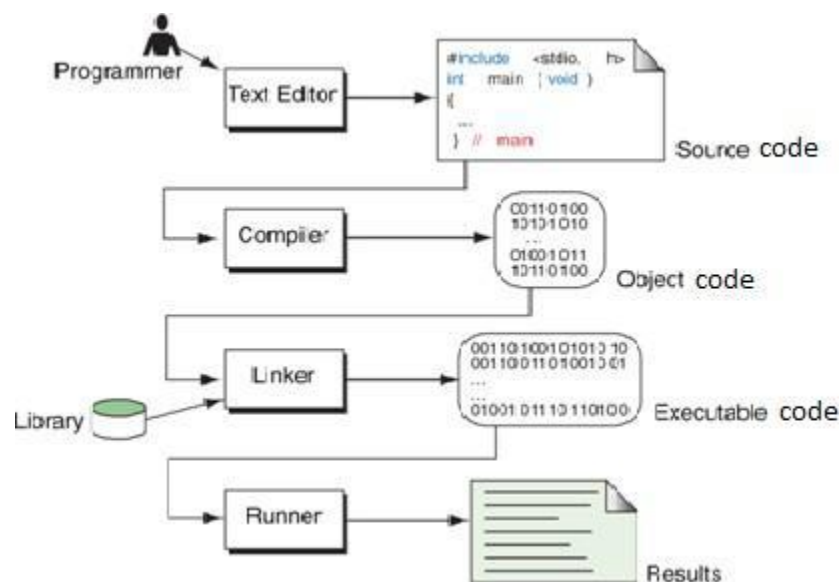


Fig: Building a C – program

B. Compiling Programs:

The code in a source file on the disk must be translated into machine language. This is the job of compiler which translates code in source file stored on disk into machine language. The C compiler is actually two separate programs: the pre-processor and the translator. The pre-processor reads the source code and prepares it for the compiler. It scans special instructions known as pre-processor commands. These commands tell the pre-processor to take for special code libraries, make substitutions in the code. The result of pre-processing is called translation unit. The translator reads the translation unit and writes resulting object module to a file that can be combined with other precompiled units to form the final program. An object module is the code in machine language. This module is not ready for execution because it does not have the required C and other functions included.

C. Linking Programs:

C programs are made up of many functions. Example: `printf()`, `cos()`... etc Their codes exists elsewhere, they must be attached to our program. The linker assembles all these functions, ours and the system's, into a final executable program.

D. Executing Programs:

Once our program has been linked, it is ready for execution. To execute a program, we use operating system command, such as `run` to load the program in to main memory and execute it. Getting program into memory is the function of an Operating System programs called loader. Loader locates the executable program and reads it into memory. In a typical program execution, the program reads data for processing, either from user or from file. After the program processes the data, it prepares output. Data output can be to user's monitor or to a file. When program has executed, Operating System removes the program from memory.

Debugging:

- Programming errors are called **bugs** and the process of tracking them down and correcting them is called **debugging**.
- Three kinds of errors can occur in a program:
 1. **Syntax errors**
 - A program can only be executed if it is syntactically correct; otherwise, the process fails and returns an error message.
 - **syntax** refers to the structure of a program and the rules about that structure.
 2. **Runtime errors**
 - So called because the error does not appear until you run the program.
 - These errors are also called **exceptions** because they usually indicate that something exceptional (and bad) has happened.
 3. **Semantic errors**
 - If there is a semantic error in the program, it will run successfully, in the sense that the computer will not generate any error messages, but it will not do the right thing. It will do something else. Specifically, it will do what the programmer told it to do.
 - But the written program does not solve the original problem. The meaning of the program (its semantics) is wrong.

Program Development:

Program Development is a multistep process that requires that we understand the problem, develop a solution, write the program, and then test it. When we are given the assignment to develop a program, we will be given a program requirements statement and the design of any program interfaces. We should also receive an overview of the complete project

so that we will take the inputs we are given and convert them to the outputs that have been specified. This is known as program design.

1. Understand the Problem

The first step in solving any problem is to understand it. By reading the requirements statements carefully, we fully understand it, we review our understanding with the user and the systems analyst to know the exact purpose.

2. Develop the solution

Once we fully understand the problem, we need to develop our solution. Three tools will help in this task. 1. Structure chart, 2. Pseudocode & 3. Flowcharts. Generally, we will use structure chart and either flowchart or Pseudo code. The structure chart is used to design the whole program. Pseudo code and flowcharts are used to design the individual parts of the program.

3. Structure chart

A structure chart, also known as hierarchy chart, shows the functional flow through our program. The structure chart shows how we are going to break our program into logical steps each step will be a separate module. The structure chart shows the interaction between all the parts (modules) of our program.

We can use flowchart or pseudo code to complete the design of your program will depend on experience and difficulty of the program your designing.

4. Write the program

When we write a program, we start with the top box on the structure chart and work our way to the bottom. This is known as top-down implementation. We will write the programs by using structure chart and flowchart or pseudo code.

5. Test the Program

Program testing can be a very tedious and time- consuming part of program development. As the programmer we are responsible for completely testing our program. In large-development projects test engineers are responsible for testing to make sure all the programs work together.

There are 2 types of testing.

1. Black box testing: This is done by the system test engineer and the user. Black box testing is the programs are tested without knowing what is inside it, without knowing how it works. Black box test plans are developed by looking only the requirements statement. The test engineer uses these requirements to develop test plans.

2. White box testing: This is the responsibility of the programmer. White box testing assumes that the tester knows everything about the program.

Except for the simplest program, one set of test data will not completely validate a program.

Introduction to the C Language

Background, C Programs, Identifiers, Data Types, Variables, Constants, Input/output functions.

Introduction to the C Language:

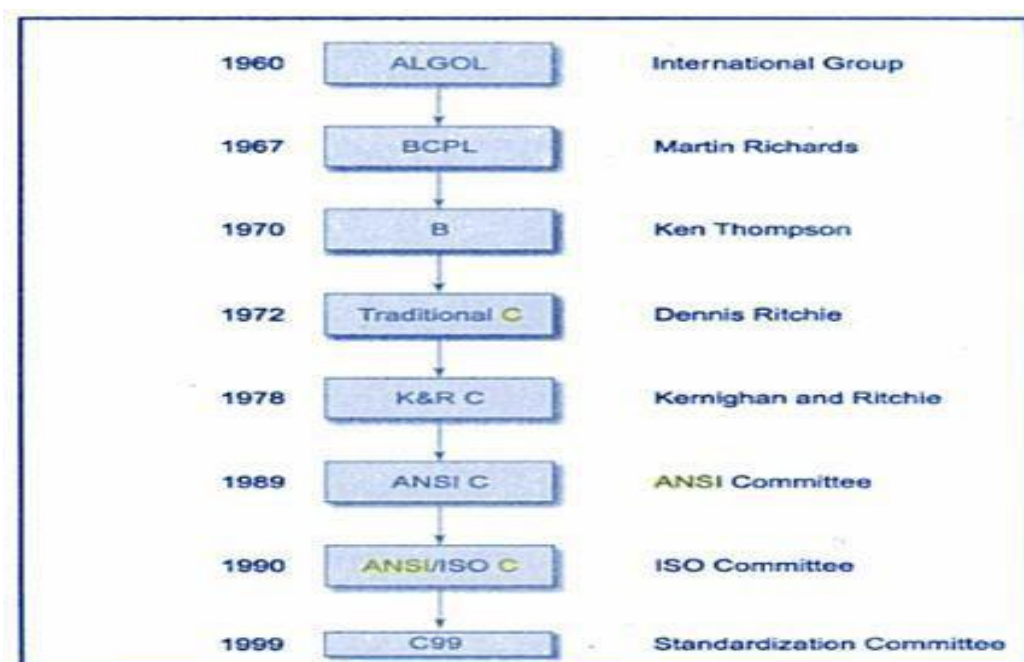
- C is a general-purpose computer programming language.
- C is also said to be structured programming language or function-oriented programming language.
- C is a High-level programming language.
- C language is used to create applications or software.
- Initially, C was developed to create an operating system called UNIX.
- The popular software like **Linux OS**, **PHP** & **MySQL** are created using C language.

Generally, C Language is used to create the following...

- Operating Systems
- Language Compilers
- Assemblers
- Interpreters
- Text Editors
- Network Drivers
- Databases

Background/ History of C Language:

The history and development of C is shown as follows



The root of all modern languages is ALGOL which is introduced in early 1960s. ALGOL was the first computer language to use a block structure. ALGOL gave the concept of **structured programming** to computer science community.

In 1967, Martin Richards developed a language called BCPL (Basic Combined Programming Language) primarily for writing system software. In 1970, Ken Thompson created a language using many features of BCPL and called it as 'B'. 'B' was used to create early versions of UNIX operating system at BELL laboratories.

'C' was evolved from ALGOL, BCPL and B by Dennis Ritchie at the Bell laboratories in 1972. The language became more popular after the publication of the book —**The C Programming Language** by **Brain Kerningham and Dennis Ritchie** in 1978. C uses many concepts from these languages and added the concept of datatypes and other powerful features. Since it was developed along with the UNIX operating system, it is strongly associated with UNIX.

For many years C was used mainly in academic environments but eventually with the release of many C compilers for commercial use it increased its popularity. This rapid growth of C led to the development of different versions of the language that were similar but often incompatible, this posed a serious problem for system developers. In order to assure that the C language remains standard in 1983 American National Standards Institute (ANSI) appointed a technical committee to define a standard for C. The committee approved a version of C in December 1989 which is now known as ANSI C which is approved by the International Standards Organization (ISO) in 1990. This version of C is referred to as C89.

During 1990's C++, a language entirely based on C, underwent a number of improvements and changes and became an ANSI/ISO approved language in November 1997. C++ added several new features to C to make it not only a true object-oriented language but also a more versatile language. Although C++ and Java were evolved out of C, the standardization committee of C felt that a few features of C++/Java, if added to C, would enhance the usefulness of the language. The result was the 1999 standard for C. This version is usually referred to as C99.

Features of C Language:

The following are the standard features of C Language as follows.

- 1. Portability:** Programs written in C language can be run on any compiler with little or no modifications of a program.
- 2. Flexibility:** C language has number of reserved keywords, which helps the programmers to take control over the usage of language and to modify the structure of program.
- 3. Modularity:** C language provides feature of modularity, i.e. we can breakdown a large program into manageable modules. This feature provides easy to understand and debugging of the program will become faster.
- 4. Robustness:** C language is a robust language, with rich set of built-in functions and operators that can be used to write any complex programs.
- 5. Extensibility:** In C language program, the new features can be added at any time by the programmer to the existing program.

6. Efficiency and effectiveness: Programs written in C language are fast and efficient, due to the variety of data types and powerful operators.

BASIC STRUCTURE OF C LANGUAGE:

The program written in C language follows this basic structure. The sequence of sections should be as they are in the basic structure. A C program should have one or more sections but the sequence of sections is to be followed.

Documentation section

Pre-Processor Directives/Linking section

Definition section

Global declaration section

Main function section

{

Local Variable Declaration section

Executable section (Instruction 1 to N)

}

Sub program or function section

Example:

```
Documentation Section
Pre-Processor Directives
Function Prototyping/Declaration
Global Variable Declaration
main Function
{
  Local Variable Declaration
  Instruction -1;
  Instruction -2;
  Instruction -3;
  --
  --
  --
  Instruction -n;
}
```

```
Sub Function Definition
{
  Local Variable Declaration
  Instruction -1;
  Instruction -2;
  --
  --
  Instruction -n;
}
```

```
/* C Program To Find Addition of Two Numbers */
#include<stdio.h>
void addition(int);
int X=100;
void main()
{
    int y=250,sum=0;
    sum = x + y;
    addition(y); //Sub Function Calling
    printf("Addition of Two number = %d\n",sum); //sum = 350
}

void addition(int y)
{
    int sum=0;
    sum = x + y;
    printf("Addition of Two number = %d\n",sum); // sum = 350
}
```

Documentation section:

This section comes first and is used to document the use of logic or reasons in your program. It can be used to write the program's objective, developer and logic details. The documentation is done in C language with `/*` and `*/`. Whatever is written between these two are called comments.

Ex: - `/*Addition of two numbers */`

`// Addition of two numbers`

Link section: Link section provides instructions to the compiler to link functions from the system library. i.e. This section tells the compiler to link the certain occurrences of keywords or functions in your program to the header files specified in this section

Ex: - `# include<stdio.h>`

Definition section: Definition section defines all symbolic constants. i.e. It is used to declare some constants and assign them some value.

Ex: - `# define A 10.`

`#define MAX 25`

Here `#define` is a compiler directive which tells the compiler whenever MAX is found in the program replace it with 25.

Global declaration section: Some of the variables that are used in more than one function throughout the program are called global variables and declared outside of all the functions. This section declares all the user-defined functions.

main () function section:

Every C program must have one main () function section. This contains two parts.

Declaration part: This part declares all the variables used in the executable part.

Ex: - `int a, b;`

Executable part: This part contains at least one statement. These two parts must appear between the opening and closing braces. The program execution begins at the opening brace and ends at the closing brace. All the statements in the declaration and executable parts end with a semicolon (;).

Sub program section: This section contains all the user-defined functions that are called in the main () function. User-defined functions are generally placed immediately after the main () function, although they may appear in any order.

SIMPLE 'C' PROGRAM - 1:

`/* simple program in c */`

`#include <stdio.h>`

`main ()`

```

{
printf ("welcome to c programming");
} /* End of main */

```

SIMPLE 'C' PROGRAM - 2:



Character set of 'C'

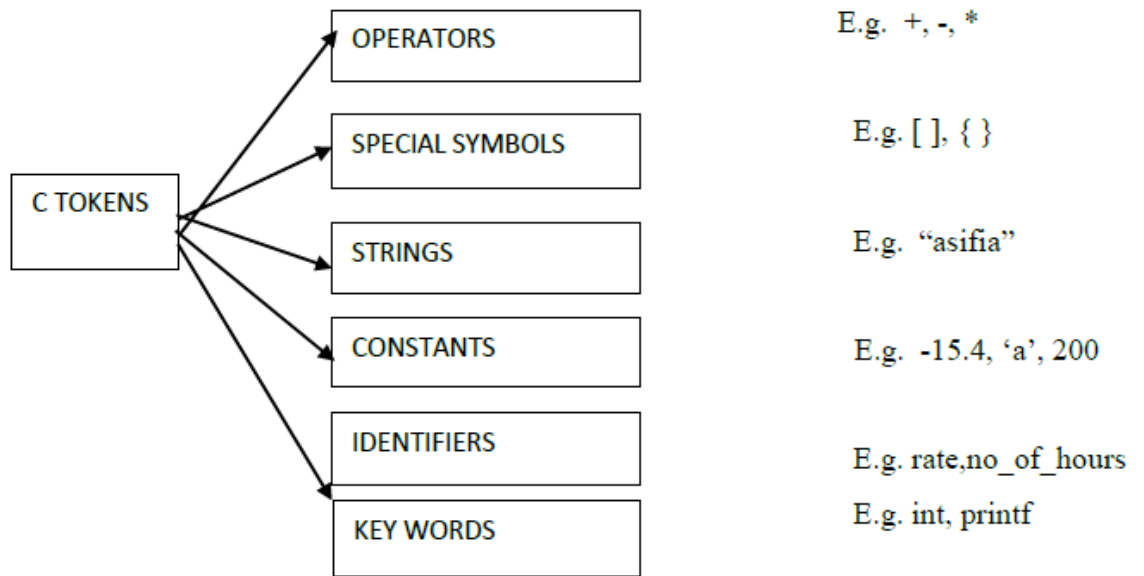
- Characters are the basic building blocks in C program, equivalent to 'letters' in English language
- Includes every printable character on the standard English language keyboard except ```, `$` and `@`

Example of characters:

- Numeric digits: 0 - 9
- Lowercase/uppercase letters: a - z and A - Z
- Space (blank)
- Special characters: `,` `.` `;` `?` `"` `/` `() [] { } * & % ^ < >` etc.,

C-TOKENS:

Tokens are individual words and punctuation marks in English language sentence. The smallest individual units are known as C tokens.



A C program can be divided into these tokens. A C program contains minimum 3 c tokens no matter what the size of the program is.

IDENTIFIERS:

- Names used for objects in C
- A 'C' program consists of two types of elements, user defined and system defined. Identifiers is nothing but a name given to these elements.
- Names of the variables and other program elements such as functions, array, etc, are known as identifiers.

There are few rules that govern the way variable are named(identifiers).

- Identifiers can consist of capital letters A to Z, the lowercase a to z, the digit 0 to 9 and underscore character_
- The first character must be a letter or an underscore.
- Maximum number of characters allowed for an identifier is 32
- No space is allowed in between the Identifiers.
- Reserved words cannot be used as identifiers.
- Special characters (Ex: ; , . , white spaces, slash etc.,) are not permitted to use as identifiers.
- Identifiers are case sensitive. Therefore, Tax and tax both different.

Recommendations for Constructing Identifiers:

1. Avoid excessively short and cryptic names such as x or wt. Instead, use a more readable and descriptive names such as student_major and down_payment.

2. Use underscores or capital letters to separate words in identifiers that consist of two or more words. Example, student_major or studentMajor are much easier to read than studentmajor.

Valid/Invalid Identifiers:

<u>Valid</u>	<u>Invalid</u>
sum	7of9
c4_5	x-name
A_NUMBER	name with spaces
longnamewithmanychars	1234a
TRUE	int
_split_name	XYZ&

After naming a variable we need to declare it to compiler of what data type it is . The format of declaring a variable is

Data-type id1, id2,. ... idn;

where data type could be float, int, char or any of the data types.

id1, id2, id3 are the names of variable we use. In case of single variable, no commas are required.

Ex: float a, b, c;
 int e, f, grand total;
 char present_or_absent;

ASSIGNING VALUES:

When we name and declare variables, we need to assign value to the variable. In some cases, we assign value to the variable directly like

a=10; in our program.

In some cases, we need to assign values to variable after the user has given input for that.

Ex: we ask user to enter any no. and input it

/* Write a program to show assigning of values to variables */

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int a;
```

```
float b;
```

```
printf("Enter any number\n");
b=190.5;
scanf("%d",&a);
printf("user entered %d", a);
printf("B's values is %f", b);
}
```

Keywords:

- Keywords are nothing but system defined identifiers.
- Keywords are reserved words of the language.
- They have specific meaning in the language and cannot be used by the programmer as variable or constant names
- C is case sensitive, it means these must be used as it is
- 32 Keywords in C Programming

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

variable:

Variable: A variable is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times. It is a way to represent memory location through symbol so that it can be easily identified. A variable may take different values at different times during execution. A variable name can be chosen by the programmer in a meaningful way so as to reflect its function or nature in the program.

Rules:

- Variable names may consist of letters, digits and under score (_) character.
- First char must be an alphabet or an _ (underscore)
- Length of the variable cannot exceed upto 8 characters, some C compilers can
- Recognized upto 31 characters.
- White space is not allowed.

- Variables name should not be a keyword.
- Uppercase and lower case are significant.

Ex: - mark, sum1, tot_value, delhi → **valid**

Prics\$, group one, char --→**invalid**

declare and initialize a variable:

Declaration does two things:

- It tells the compiler what the variable name is. It specifies what type of data the variable will hold.
- The declaration of variables must be done before they are used in the program.

The syntax for declaring a variable is as follows:

type variable_list;

data-type v1, v2,, vn;

- v1,v2,...,vn are the names of variables.
- Variables are separated by commas.
- A declaration statement must end with a semicolon.

For example, valid declarations are:

- int count;
- int number, total;
- double ratio;

The simplest declaration of a variable is shown in the following code fragment:

Ex:

```
/*.....Program Name... ..*/
main() {
/*.....Declaration .....*/
float x,y;
int code;
shortint count;
long int amount;
double deviation;
unsigned n;
char c;
/*.....Computation .....*/
}/*.....Program ends .....*/
```

Initialization of variable: Initialize a variable in C is to assign it a starting value. Without this we can't get whatever happened to memory at that moment. C does not initialize variables automatically. So, if you do not initialize them properly, you can get unexpected results. Fortunately, C makes it easy to initialize variables when you declare them.

For Example:

- `int x=45;`
- `int month_lengths[] = {23,34,43,56,32,12,24};`
- `struct role = {"Hamlet", 7, FALSE, "Prince of Denmark ", "Kenneth Branagh"};`

Note: The initialization of variable is a good process in programming.

There are many types of variables in C language, Two main are,

Local variable

Global variable

Local variable in C:

- The scope of local variables will be within the function only.
- These variables are declared within the function and can't be accessed outside the function.

Global variable in C:

- The scope of global variables will be throughout the program. These variables can be accessed from anywhere in the program.
- This variable is defined outside the main function. So that, this variable is visible to main function and all other sub functions.

DATA TYPES:

Data types

Data type is the type of the data that are going to access within the program. C supports different data types. Each data type may have pre-defined memory requirement and storage representation. C supports 4 classes of data types.

1. Primary or fundamental data type (int, char, float, double): Primitive data types are the most basic data types that are used for representing simple values such as integers, float, characters, etc.

2. Empty data type (void)- void type has no value.

3. Derived data type (arrays, pointers): The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.

4. User-defined data type (structures, unions, Enumeration, typedef): The user-defined data types are defined by the user himself.

1byte = 8 bits (0_s and 1_s)

1. Primary or (fundamental) data type

All C compilers support 4 fundamentals data types, they are

- i) Integer (int)
- ii) Floating (float)
- iii) Double – precision floating point(double)
- iv) Character(char)

i). Integer types:

- The integer datatype in C is used to store the integer numbers (any number including positive, negative and zero without decimal part).
- Octal values, hexadecimal values, and decimal values can be stored in int data type in C.
- Integers are whole numbers with a range of values supported by a particular machine.
- Integers occupy one word of storage and since the word size of the machine vary.
- If we use 16-bit word length the size of an integer value is -32768 to +32767.
- In order to control over the range of numbers and storage space, C has 3 classes of integer storage, namely short, long, and unsigned.

Syntax of Integer

We use int keyword to declare the integer variable:

int var_name;

Data Type	Size (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd
signed short int	2	Same as short int	%hd
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
int	4	-2,147,483,648 to 2,147,483,647	%d, %i
signed int	4	(same as int)	%d, %i
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	-(2 ⁶³) to (2 ⁶³)-1	%lld
unsigned long long int	8	0 to 18,446,744,073,709,551,615	%llu

Example-1

```
#include <stdio.h>
```

```
void main()
```

```
{
int i = 5;
printf("The integer value is: %d \n", i);
}
```

Example-2

// C program to print Integer data types.

```
#include <stdio.h>

int main ()
{
    // Integer value with positive data.
    int a = 9;

    // integer value with negative data.
    int b = -9;

    // U or u is Used for Unsigned int in C.
    int c = 89U;

    // L or l is used for long int in C.
    long int d = 99998L;

    printf("Integer value with positive data: %d\n", a);
    printf("Integer value with negative data: %d\n", b);
    printf("Integer value with an unsigned int data: %u\n", c);
    printf("Integer value with an long int data: %ld", d);

    return 0;
}
```

ii) Real Data Type:

In C programming Real data type is used to store floating-point values and decimal and exponential values.

Refers to all the real number values or decimal points

Floating point types:

- The float data type is used to store fractional numbers (real numbers) with 6 digits of precision.
- Floating point numbers are denoted by the keyword float.

float var_name

Double precision type:

- When the accuracy of the floating-point number is insufficient, we can use the double to define the number. The double is same as float but with longer precision and takes double space (8 bytes) than float.
- To extend the precision further we can use long double which occupies 16 bytes of memory space.

Data Type	Size (bytes)	Range	Format Specifier	Precision
float	4	1.2E-38 to 3.4E+38	%f	6 Decimal Places
double	8	1.7E-308 to 1.7E+308	%lf	15 Decimal Places
long double	16	3.4E-4932 to 1.1E+4932	%Lf	19 Decimal Places

Example:

```
#include <stdio.h>

void main ()
{
    float f = 7.2357;
    double d = 71.2357455;
    printf("The float value is: %f \n", f);
    printf("The double value is: %lf \n", d);
}
```

OUTPUT

The float value is: 7.235700

The double value is: 71.235745

iv) Character Data Type:

- Character data type allows its variable to store only a single character.
- It stores a single character; the size of the character is 1 byte.
- It is the most basic data type in C.
- Two classes of char types are there. signed char, unsigned char.

The char keyword is used to declare the variable of character type:

char var_name;

Example:

```
#include <stdio.h>

void main() {
    char c = 'b';
    printf("The character value is: %c \n", c);
}
```

Data Type	Size (bytes)	Range	Format Specifier
char or signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c

2. Void Data Type:

- void type means no value. Void is used in multiple ways as function return type, function arguments as void, and pointers to void.

Syntax:

```
// function return type void
void exit(int check);

// Function without any parameter can accept void.
int print(void);

// memory allocation function which returns a pointer to void.
void *malloc (size_t size);
```

3. Derived data types

Derived data types are primary data types that are grouped together.

You can group many elements of similar data types.

array:

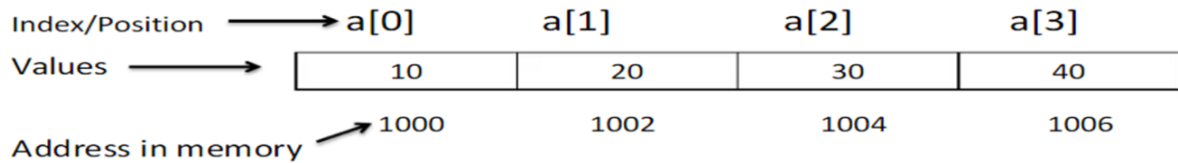
- An array in C is a collection of multiple values of a similar data type and is stored in a contiguous memory location. An array can consist of chars, integers, doubles, etc.

Declaration of Array in C

```
data_type array_name[array_size];
```

e.g.

```
int a[4]={10,20,30,40};
```



Pointer:

- C Pointer is a special variable that can be used to store address of another variable
- pointer is a variable whose value is the address of another value. i.e. direct address of the memory location.

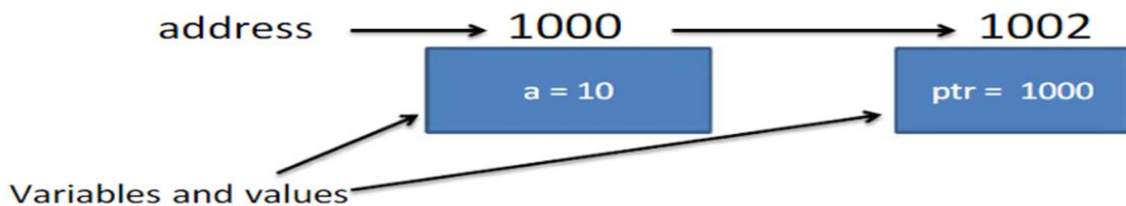
The general form of a pointer variable declaration is -

datatype *var-name;

- The asterisk * used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer.

e.g. `int a=10, *ptr;`

`ptr=&a;`



User-Defined Data Types

A user - defined data type (UDT) is a data type that derived from an existing data type but they have their own keywords too along with the use of basic data types.

1. structure
2. union
3. enum
4. typedef

structure:

- It is a package of variables of different types under a single name.
- "struct" keyword is used to define a structure.
- The total bytes of data types occupy the memory.

e.g.

```

struct student
{
    int rollno;
    char name[10];
    float per;
} std;

```

Structure student will occupy $2+1+4=7$ bytes in memory.

Union

- It is a package of variables of different types under a single name.
- “union” keyword is used to define a structure.
- The highest bytes of data types will occupy the memory.

e.g.

```

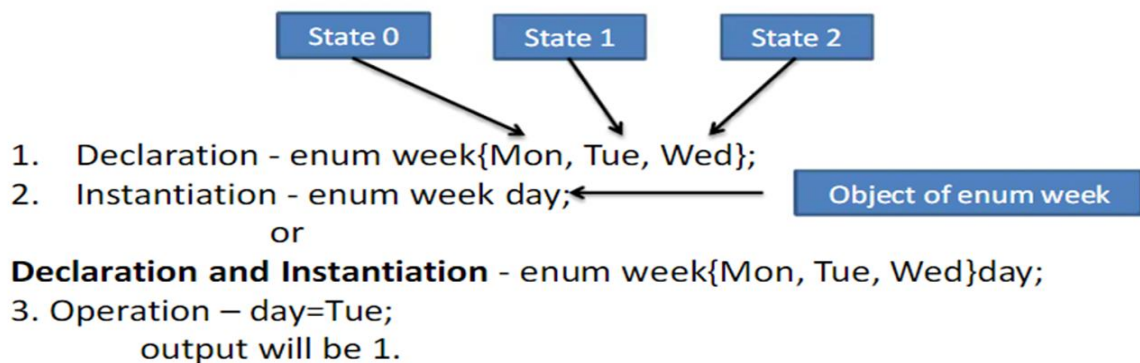
union student
{
    int rollno;
    char name[10];
    float per;
} std;

```

- Structure student will occupy 4 bytes in memory as float is the highest data type among int and char

Enumeration:

- Enumeration is a special data type that consists of integral constants, and each of them is assigned with a specific name.
- “enum” keyword is used to define the enumerated data type.



Example:


```
#include <stdio.h>

enum week {Mon, Tue, Wed, Thur, Fri, Sat, Sun};

void main()
{
enum week day;
day = Tue;
printf("%d",day);
}
```

output -1

Typedef

- You can define explicitly new data type names by using the keyword typedef
- Using typedef does not actually create a new data class, rather it defines a name for an existing type.

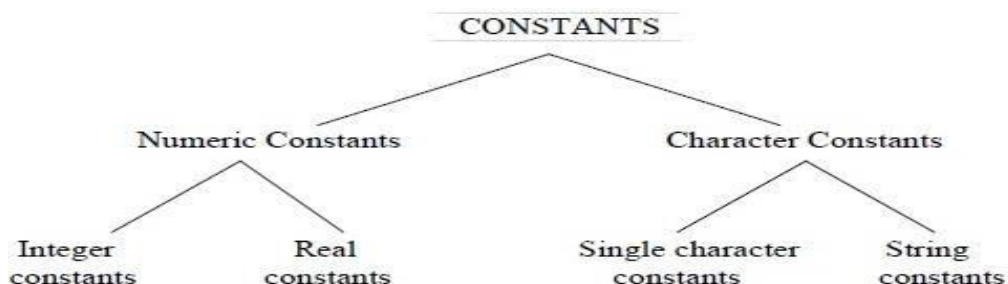
```
void main()
{
typedef int suman;
suman a,b; //a and b are of int data type
-----
}
```

- Now, suman will act as int data type.

Constants:

- C Constants are like normal variables. But only difference is their values cannot be modified by the program once they are defined.
- Constants refer to fixed values. They are also called as literals.
- Constants may be belonging to any of the data type.

Syntax: **const** data_type variable_name; (or) **const** data_type *variable_name;



Integer Constants:

An integer constant refers to a sequence of digits. There are three types of integers, namely, decimal integer, octal integer and hexadecimal integer. A prefix specifies the base or radix: 0x or 0X for hexadecimal, 0 for octal, and nothing for decimal. Some examples are:

426, +786, -34(decimal integers)

037, 0345, 0661(octal integers)

0X2, 0X9F, 0X (hexadecimal integers)

30u /* unsigned int */

30l /* long */

30ul /* unsigned long */

Rules for constructing Integer constants:

1. An integer constant must have at least one digit.
2. It must not have a decimal point.
3. It can be either positive or negative.
4. If no sign precedes an integer constant, it is assumed to be positive.
5. Commas or blanks are not allowed within an integer constant.

Example: -33 32767

There are **three** types of integer constants namely:

- a) Decimal integer constant
- b) Octal integer constant
- c) Hexadecimal integer constant

Decimal Integer constant (base 10):

- It consists of any combinations of digits taken from the set **0** through **9**, preceded by an optional – or + sign.
- The first digit must be other than **0**.
- White spaces, commas, and non-digit characters are not permitted between digits.

Valid: 32767 -9999 -23

Invalid: 12,245 -Illegal character (,)

10 20 30 -Illegal character (blank space)

b) Octal Integer Constant (base 8):

- It consists of any combinations of digits taken from the set **0** through **7**.
- If a constant contains two or more digits, the first digit must be 0.
- In programming, octal numbers are used.

Valid: 037 0435

Invalid: 0786 - Illegal digit 8

123 - Does not begin with zero

01.2 -Illegal character (.)

c) Hexadecimal integer constant (base 16):

- It consists of any combinations of digits taken from the set **0** through **9** and also **a** through **f** (either **uppercase** or **lowercase**).
- The letters **a** through **f** (or A through F) represent the decimal quantities **10** through **15** respectively.
- This constant must begin with either **0x** or **0X**.

Valid: 0x 0X1 0x7F

Invalid: 0xefg - Illegal character **g**

123 - Does not begin with 0x

Real (or) floating Constants:

A floating-point literal has an integer part, a decimal point, a fractional part, and an exponent part. You can represent floating point literals either in decimal form or exponential form.

Some examples are:

3.14159 /* Legal */

314159E-5L /* Legal */

510E /* Illegal: incomplete exponent */

210f /* Illegal: no decimal or exponent */

.e55 /* Illegal: missing integer or fraction */

The exponential form of representation of real constants is usually used if the value of the constant is either too small or too large. In exponential form of representation, the real constant is represented in two parts. The first part present before 'e' is called **Mantissa** and the part following 'e' is called **Exponent**.

For ex. 0.000342 can be written in Exponential form as 3.42e-4.

Rules of floating constants:

- A real constant must have at least one digit
- It must have a decimal point
- It could be either positive or negative
- If no sign precedes an integer constant, it is assumed to be positive.
- No commas or blanks are allowed within a real constant.

Single Character Constants:

- Single character constant contains a single character enclosed within a pair of single quote marks.
- For ex. 'A', '5', ';', ''''
- Note that the character constant '5' is not same as the number 5. The last constant is a blank space.
- Character constant has integer values known as ASCII values.

For example, the statement `printf ("%d", a);` would print the number 97, the ASCII value of the letter a. Similarly, the statement `printf ('%c',97);` would output the letter 'a'.

Backslash or escape sequence Character Constants:

C supports some special backslash character constants that are used in output functions. Some of the back-slash character constants are as follows:

- There are some characters which have special meaning in C language.
- They should be preceded by **backslash symbol** to make use of special function of them.
- Given below is the list of special characters and their purpose.

Constant	Meaning
'\0'	Null
'\t'	Horizontal tab
'\b'	Back space
'\a'	Audible alert
'\f'	Form feed
'\n'	New line
'\r'	Carriage return
'\v'	Vertical tab
'\''	Single quote
'\"'	Double quote
'\?'	Question mark
'\\'	backslash

String constants:

A string constant is a sequence of characters enclosed in double quotes. The characters may be letters, numbers, special characters and blank space.

Example: " Hello!"

 "1986"

Input/output functions:

Input: In any programming language input means to feed some data into program. This can be given in the form of file or from command line.

Output: In any programming language output means to display some data on screen, printer or in any file.

We have the following categories of IO function in C –

- **Unformatted character IO functions:** getchar() and putchar()
- **Unformatted string IO functions:** gets() and puts()
- **Formatted IO functions:** scanf() and printf()

The unformatted I/O functions read and write data as a stream of bytes without any format, whereas formatted I/O functions use predefined formats like "%d", "%c" or "%s" to read and write data from a stream.

Unformatted Character Input & Output Functions: getchar() and putchar()

These two functions accept a single character as input from the keyboard, and display a single character on the output terminal, respectively.

The getchar() function it reads a single key stroke without the Enter key.

int getchar(void)

There are no parameters required. The function returns an integer corresponding to the ASCII value of the character key input by the user.

Example 1

The following program reads a single key into a char variable –

```
#include <stdio.h>
```

```
int main() {
```

```
    char ch;
```

```
    printf("Enter a character: ");
```

```
    ch = getchar();
```

```
    puts("You entered: ");
```

```
    putchar(ch);
```

```
    return 0;
```

```
}
```

Output:

Enter a character: W

You entered:

W

You can also use the **unformatted putchar() function** to print a single character. The C library function "**int putchar(int char)**" writes a character (an unsigned **char**) specified by the argument char to **stdout**.

int putchar(int c)

A single parameter to this function is the character to be written. You can also pass its ASCII equivalent integer. This function returns the character written as an unsigned char cast to an int or EOF on error.

Example:

The following example shows how you can use the putchar() function –

```
#include <stdio.h>

int main() {
    char ch;
    for(ch = 'A' ; ch <= 'Z' ; ch++) {
        putchar(ch);
    }
    return(0);
}
```

Output

ABCDEFGHIJKLMNOPQRSTUVWXYZ

UnFormatted String Input & Output Functions: gets(), fgets(), puts(), and fputs()

The **char *gets(char *str)** function reads a line from **stdin** into the buffer pointed to by str until either a terminating newline or EOF (End of File) is encountered.

char *gets (char *str)

This function has a single argument. It is the pointer to an array of chars where the [C string](#) is stored. The function returns "str" on success and NULL on error or when EOF occurs while no characters have been read.

Example:

```
#include <stdio.h>

int main() {
    char name[20];
    printf("Enter your name: ");
```

```
    gets(name);  
    printf("You entered the name: %s", name);  
    return 0;  
}
```

Output

Enter your name: Ravikant

You entered the name: Ravikant

fgets() Function

In newer versions of C, **gets()** has been deprecated as it is potentially a dangerous function because it doesn't perform bound checks and may result in buffer overflow.

Instead, it is advised to use **fgets()** function –

fgets(char arr[], size, stream);

fgets() can be used to accept input from any input stream such as stdin (keyboard) or FILE (file stream).

Example:

```
#include <stdio.h>  
  
int main () {  
    char name[20];  
    printf("Enter a name: \n");  
    fgets(name, sizeof(name), stdin);  
    printf("You entered: \n");  
    printf("%s", name);  
    return 0;  
}
```

Output:

Enter a name:

Rakesh Sharma

You entered:

Rakesh Sharma

The function "**int puts (const char *str)**" writes the string 's' and a trailing newline to **stdout**.

int puts(const char *str)

The str parameter is the C string to be written. If successful, it returns a non-negative value. On error, the function returns EOF.

We can use the **printf()** function with %s specifier to print a string. We can also use the **puts()** function (deprecated in C11 and C17 versions) or the **fputs()** function as an alternative.

Example:

```
#include <stdio.h>

int main () {
    char name[20] = "Rakesh Sharma";
    printf("With puts(): \n");
    puts(name);
    printf("\nWith fputs(): \n");
    fputs(name, stdout);
    return 0;
}
```

Output:

With puts():

Rakesh Sharma

With fputs():

Rakesh Sharma

Formatted Input & Output Functions: scanf() and printf()

The scanf() function reads the input from the standard input stream **stdin** and scans that input according to the format provided –

int scanf(const char *format, ...)

The printf() function writes the output to the standard output stream stdout and produces the output according to the format provided.

int printf(const char *format, ...)

The format can be a simple constant string, but you can specify %s, %d, %c, %f, etc., to print or read strings, integers, characters or floats respectively. There are many other formatting options available which can be used based on the specific requirements.

Character testing functions:

- "ctype.h" library for analyzing the character input and converting them.

Function	Checks whether entered character is
isalpha	An alphabet (or) not
isdigit	A digit (or) not
isspace	A space, a newline (or) tab
ispunct	A special symbol (or) not
islower	A lower case letter of alphabet
isupper	An upper case letter of alphabet
isalphanumeric	An alphabet/digit or not

Example:

```
#include <stdio.h>
#include <ctype.h>
main(){
    char character;
    printf("Press any key digit or alphabet");
    character = getchar();
    if (isalpha(character) )
        printf("The character is a letter.");
    else
        if (isdigit (character) )
            printf("The character is a digit.");
        else
            printf("The character is not alphanumeric.");
}
```

Case 1:

Press any key digit or alphabet

3

The character is a digit.

Case 2:

Press any key digit or alphabet

G

The character is a letter.

Operators

Arithmetic, relational, logical, bitwise, conditional, increment/decrement, assignment, C program examples. Expressions, Precedence and Associativity, Expression Evaluation, Type conversions.

Operators:

An operator is a symbol (like +, -, *, /) that directs the computer to perform certain mathematical or logical manipulations and is usually used to manipulate data and variables”

Ex: a+b

C operators can be classified into a number of categories, they are

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increment and decrement operators
6. Conditional operators
7. Bitwise Operators
8. Special operators

1. Arithmetic Operators: C provides all the basic arithmetic operators. These can operate on any built –in data type allowed in C.

The arithmetic operators are listed as follows:

Operator	example	Meaning
+	a + b	Addition –unary
-	a – b	Subtraction- unary
*	a * b	Multiplication
/	a / b	Division
%	a % b	Modulo division- remainder

Integer division truncates any fractional part. The modulo division operation produces the remainder of an integer division.

Ex: Here a and b are operands, let a=14 and b=4 we have the following results

$a-b = 10$

$a+b = 18$

$a*b = 56$

$a/b = 3(\text{coefficient})$

$a\%b = 2(\text{remainder})$

Example:

// Working of arithmetic operators

```
#include <stdio.h>

int main()
{
    int a = 9, b = 4, c;
    c = a+b;
    printf("a+b = %d \n", c); //13
    c = a-b;
    printf("a-b = %d \n", c); //5
    c = a*b;
    printf("a*b = %d \n", c); // 36
    c = a/b;
    printf("a/b = %d \n", c); // 2
    c = a%b;
    printf("Remainder when a divided by b = %d \n", c); // 1
    return 0;
}
```

2. Relational Operators:

Relational operators are used for comparing two quantities, and take certain decisions. For example, we may compare the age of two persons or the price of two items.... these comparisons can be done with the help of relational operators. An expression containing a relational operator is termed as a relational expression. The value of a relational expression is either one or zero. It is one if the specified relation is true and zero if the relation is false.

Ex: - 13<34 (true) 23>35(false)

C supports 6 relational operators

Relational Operator	Meaning of Operator	Example
==	Equal to	5 == 3 is evaluated to 0
>	Greater than	5 > 3 is evaluated to 1
<	Less than	5 < 3 is evaluated to 0
!=	Not equal to	5 != 3 is evaluated to 1
>=	Greater than or equal to	5 >= 3 is evaluated to 1
<=	Less than or equal to	5 <= 3 is evaluated to 0

When arithmetic expression is used on either side of a relational operator, the arithmetic expression will be evaluated first and then the results compared, that means arithmetic operators have a higher priority over relational operators.

Example:

// Working of relational operators

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 5, b = 5, c = 10;
```

```
    printf("%d == %d is %d \n", a, b, a == b); //5 == 5 is 1
```

```
    printf("%d == %d is %d \n", a, c, a == c); // 5 == 10 is 0
```

```
    printf("%d > %d is %d \n", a, b, a > b); // 5 > 5 is 0
```

```
    printf("%d > %d is %d \n", a, c, a > c); // 5 > 10 is 0
```

```
    printf("%d < %d is %d \n", a, b, a < b); // 5 < 5 is 0
```

```
    printf("%d < %d is %d \n", a, c, a < c); // 5 < 10 is 1
```

```
    printf("%d != %d is %d \n", a, b, a != b); // 5 != 5 is 0
```

```
    printf("%d != %d is %d \n", a, c, a != c); // 5 != 10 is 1
```

```

printf("%d >= %d is %d \n", a, b, a >= b); // 5 >= 5 is 1
printf("%d >= %d is %d \n", a, c, a >= c); // 5 >= 10 is 0
printf("%d <= %d is %d \n", a, b, a <= b); // 5 <= 5 is 1
printf("%d <= %d is %d \n", a, c, a <= c); // 5 <= 10 is 1

return 0;

}

```

3. Logical Operator:

C has 3 logical operators. The logical operators are used when we want to test more than one condition and make decisions. The operators are as follows:

Operator	Meaning	Example
&&	Logical AND. True only if all operands are true	If c = 5 and d = 2 then, expression ((c==5) && (d>5)) equals to 0.
	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression ((c==5) (d>5)) equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression !(c==5) equals to 0.

The logical operators && and || are used when we test more than one condition and make decisions. Logical expression or a compound relational expression- An expression that combines two or more relational expressions

Ex: if (a==b && b==c)

Ex: - a>b && x==45

Truth Table:

a	b	Value of the expression	
		a && b	a b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

This expression combines two or more relational expressions, is termed as a logical expression or a compound relational expression. The logical expression given above is true only if $a > b$ is true and $x == 10$ is true. If both of them are false the expression is false. The truth table for logical and & logical or operators are shown in above table.

Some examples of logical expression

1. if (age > 55 && salary < 1000)
2. if (number < 0 || number > 100)

Example:

// Working of logical operators

```
#include <stdio.h>

int main()
{
    int a = 5, b = 5, c = 10, result;
    result = (a == b) && (c > b);
    printf("(a == b) && (c > b) is %d \n", result); // (a == b) && (c > b) is 1
    result = (a == b) && (c < b);
    printf("(a == b) && (c < b) is %d \n", result); // (a == b) && (c < b) is 0
    result = (a == b) || (c < b);
    printf("(a == b) || (c < b) is %d \n", result); // (a == b) || (c < b) is 1
    result = (a != b) || (c < b);
    printf("(a != b) || (c < b) is %d \n", result); // (a != b) || (c < b) is 0
    result = !(a != b);
    printf("(a != b) is %d \n", result); // (a != b) is 1
    result = !(a == b);
    printf("(a == b) is %d \n", result); // (a == b) is 0
    return 0;
}
```

4. Assignment operator:

These operators are used to assign the result of an expression to a variable. The usual assignment operator is “=”. In addition, C has a set of shorthand assignment operators of the form:

v op = exp;

Where v is a variable, exp is an expression and op is a C binary arithmetic operator. The operator op= is known as the shorthand assignment operator.

The assignment statement `v op=exp;` is equivalent to `v =v op (exp);`

Ex: - `x+=y+1;` is equivalent to `x= x+(y+1);` `a*=a;` is equivalent to `a=a*a;`

The use of short hand assignment operators has the following advantages:

- What appears on the left-hand side need not be repeated and therefore it becomes easier to write
- The statement is more concise and easier to read
- The statement is more efficient

Assignment Operator	Example	Same as
=	<code>a = b</code>	<code>a = b</code>
+=	<code>a += b</code>	<code>a = a+b</code>
-=	<code>a -= b</code>	<code>a = a-b</code>
*=	<code>a *= b</code>	<code>a = a*b</code>
/=	<code>a /= b</code>	<code>a = a/b</code>
%=	<code>a %= b</code>	<code>a = a%b</code>

Example:

// Working of assignment operators

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 5, c;
```

```
    c = a;
```

```
    printf("c = %d\n", c); // c = 5
```

```
    c += a;
```

```
    printf("c = %d\n", c); // c = 10
```

```
    c -= a;
```

```
    printf("c = %d\n", c); // c = 5
```

```
    c *= a;
```

```

printf("c = %d\n", c); // c = 25

c /= a;

printf("c = %d\n", c); // c = 5

c %= a;

printf("c = %d\n", c); // c = 0

return 0;

}

```

5. Increment and Decrement operators:

++ and -- are increment and decrement operators in C. The operator ++ adds 1 to the operand, while -- subtracts 1. Both are unary operators.

++m; or m++; is equal to m=m+1(m+=1;)

--m; or m- - is equal to m=m-1(m- = 1;)

We use the increment and decrement statements in for and while loops extensively.

Rules for ++ & -- operators

1. These require variables as their operands
2. When postfix either ++ or -- is used with the variable in a given expression, the expression is evaluated first and then it is incremented or decremented by one
3. When prefix either ++ or -- is used with the variable in a given expression, it is incremented or decremented by one first and then the expression is evaluated with the new value

Ex: - m=5;
 y=++m;
 the value of y=6 and m =6.

Suppose if we write the above statement as

```

m=5;

y= m++;

the value of y=5 and m=6.

```

A prefix operator first adds 1 to the operand and then the result is assigned to the variable on left. On the other hand, a postfix operator first assigns the value to the variable on left and then increments the operand.

Example:

// Working of increment and decrement operators

```
#include <stdio.h>

int main()
{
    int a = 10, b = 100;
    float c = 10.5, d = 100.5;
    printf("++a = %d \n", ++a); // ++a = 11
    printf("--b = %d \n", --b); // --b = 99
    printf("++c = %f \n", ++c); // ++c = 11.500000
    printf("--d = %f \n", --d); // --d = 99.500000
    return 0;
}
```

6. Conditional operator:

A ternary operator pair “?:” is available in C to construct conditional expressions of the form

exp1? exp2: exp3 Where exp1, exp2 and exp3 are expressions,

The operator ?: works as follows: exp1 is evaluated first. If it is non-zero (true), then the expression exp 2 is evaluated and becomes the value of the expression. If exp1 is false, exp3 is evaluated and its value becomes the value of the expression.

Ex: - a=10; b=45;

X = (a>b) ?a:b;

o/p: - X will be assigned the value of b (45).

7. Bitwise Operators:

C supports a special operator known as bitwise operators for manipulation of data at bit level. These operators are used for testing the bits, or shifting them right to left. Bitwise operators may not be applied to float or double.

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
<<	Shift left
>>	Shift right
~	Bitwise complement

The truth tables for **&**, **|**, and **^** are as follows –

p	q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume A = 60 and B = 13 in binary format, they will be as follows –

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) = 12, i.e., 0000 1100
 	Binary OR Operator copies a bit if it exists in either operand.	(A B) = 61, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) = 49, i.e., 0011 0001
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A) = ~(60), i.e., -1100 0011
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 = 240 i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 = 15 i.e., 0000 1111

8. Special operators:

C supports some special operators such as comma operator, size of operator, pointer operators (& and *) and member selection operators (. and ->).

comma operator:

The comma operator is used to link the related expressions together. A comma- linked list of expressions is evaluated left to right and the value of right- most expression is the value of the combined expression. For example, the statement

```
value = (x=10, y=5, x+y);
```

This statement first assigns the value 10 to x, then assigns 5 to y and finally assigns 15. In for loops:

```
for (n=1 , m=10, n<=m; n++, m++);
```

sizeof operator:

The sizeof is a compile time operator and when used with an operand, it returns the number of bytes the operand occupies. The operand may be variable, a constant or a data type qualifier.

```
m = sizeof (sum); n = sizeof (long int);
```

The sizeof operator is normally used to determine the lengths of arrays and structures when their sizes are not known to the programmer. It is also used to allocate memory space dynamically to variables during execution of a program.

Example:

```
#include <stdio.h>

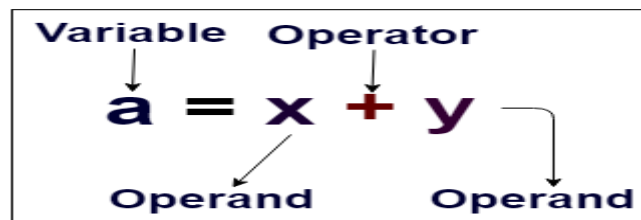
int main()
{
    int a;
    float b;
    double c;
    char d;

    printf("Size of int=%lu bytes\n",sizeof(a)); // Size of int = 4 bytes
    printf("Size of float=%lu bytes\n",sizeof(b)); // Size of float = 4 bytes
    printf("Size of double=%lu bytes\n",sizeof(c)); // Size of double = 8 bytes
    printf("Size of char=%lu byte\n",sizeof(d)); // Size of char = 1 byte

    return 0;
}
```

Expressions:

An expression in C is some combination of constants, variables, operators and function calls. Sample expressions are:



- Most expressions have a value based on their contents.
- A statement in C is just an expression terminated with a semicolon.

For example: `sum = x + y + z; printf ("Go Buckeyes!");`

The rules given below are used to evaluate an expression,

- If an expression has parenthesis, sub expression within the parenthesis is evaluated first and arithmetic expression without parenthesis is evaluated first.
- The operators of high-level precedence are evaluated first.
- The operators at the same precedence are evaluated from left to right or right to left depending on the associativity of operators.

Expressions are evaluated using an assignment statement of the form: `variable = expression;` variable is any valid C variable name. When the statement is encountered, the expression is evaluated first and the result then replaces the previous value of the variable on the left-hand side. All variables used in the expression must be assigned values before evaluation is attempted.

Ex: `- x = a*b-c;      y = b/c*a;      z = a-b / c+d;`

Ex: `- x = a-b/3+c*2-1` when `a=9`, `b=12`, and `c=3` the expression becomes.

$$x = 9 - 12/3 + 3*2 - 1$$

Step1: `x = 9 - 4 + 3*2 - 1`

Step2: `x = 9 - 4 + 6 - 1`

Step3: `x = 5 + 6 - 1`

Step4: `x = 11 - 1`

Step5: `x = 10`

An expression can be defined depending on the position and number of its operator and operands:

1. **Infix Expression** (operator is used *between* the operands)

$$a = x + y$$

2. **Postfix Expression** (operator is used *after* the operands)

xy+

3. Prefix Expression (operator is used before the operands)

+xy

Types of Expressions:

- **Arithmetic expressions:** It consists of arithmetic operators (+, -, *, and /) and computes values of int, float, or double type.

Example: #include <stdio.h>

```
int main(){
//Arithmetic Expression
int a = (6 * 2) + 7 - 9;
printf("The arithmetic expression returns: %d\n", a);
return 0; }
```

- **Relational expressions:** It usually consists of comparison operators (> , < , >= , <= , == , and !=) and computes the answer in the bool type, i.e., true (1) or false (0).

Example: #include <stdio.h>

```
int main(){
//Relational Expression
int b = 10;
printf("The relational expression returns: %d\n", b % 2 == 0);
return 0; }
```

- **Logical expressions:** It consists of logical operators (&&, ||, and !) and combines relational expressions to compute answers in the bool type.

Example: #include <stdio.h>

```
int main(){
//Logical Expression
int c = (7 > 9) && ( 5 <= 9);
printf("The logical expression returns: %d\n", c);
return 0; }
```

- **Conditional expressions:** It consists of conditional statements that return true if the condition is met and false otherwise.

Example: #include <stdio.h>

```
int main(){
```

```
//Conditional Expression

int d = (34 > 7) ? 1 : 0;

printf("The conditional expression returns: %d\n", d);

return 0;

}
```

Expression Evaluation:

- In the C programming language, an expression is evaluated based on the operator precedence and associativity. When there are multiple operators in an expression, they are evaluated according to their precedence and associativity. The operator with higher precedence is evaluated first and the operator with the least precedence is evaluated last.

Operator Precedence (priority).

- Operator precedence is used to determine the order of operators evaluated in an expression.

Operator Associativity

- Operator associativity is used to determine the order of operators with equal precedence evaluated in an expression.

Category	Operator	Associativity
Postfix	() [] -> .	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

Example 1: Evaluate $6*2/(2+1*2/3+6)+8*(8/4)$

Evaluation of expression	Description of each operation
$6*2/(2+1*2/3+6)+8*(8/4)$	An expression is given.
$6*2/(2+2/3+6)+8*(8/4)$	1 is multiplied by 2, giving value 2.
$6*2/(2+0+6)+8*(8/4)$	2 is divided by 3, giving value 0.
$6*2/8+8*(8/4)$	2 is added to 6, giving value 8.
$6*2/8+8*2$	8 is divided by 4, giving value 2.
$12/8+8*2$	6 is multiplied by 2, giving value 12.
$1+8*2$	12 is divided by 8, giving value 1.
$1+16$	8 is multiplied by 2, giving value 16.
17	1 is added to 16, giving value 17.

Example 2: Evaluate $x1 = (-b + \sqrt{b*b-4*a*c})/(2*a)$ @ $a=1, b=-5, c=6$

$$\begin{aligned}
 &\text{Evaluate } x1 = (-b + \sqrt{b*b-4*a*c})/(2*a) \text{ @ } a=1, b=-5, c=6 \\
 &= -(-5) + \sqrt{(-5)(-5) - 4*1*6} / (2*1) \\
 &= (5 + \sqrt{(-5)(-5) - 4*1*6}) / (2*1) \\
 &= (5 + \sqrt{25 - 4*1*6}) / (2*1) \\
 &= (5 + \sqrt{25 - 4*6}) / (2*1) \\
 &= (5 + \sqrt{25 - 24}) / (2*1) \\
 &= (5 + \sqrt{1}) / (2*1) \\
 &= (5 + 1.0) / (2*1) \\
 &= (6.0) / (2*1) \\
 &= 6.0 / 2 = 3.0
 \end{aligned}$$

Example 3: Evaluate the expression when $a=4, b=a-++a$

$$\begin{aligned}
 b = a - ++a &= a - 5 \\
 &= 5 - 5 \\
 &= 0
 \end{aligned}$$

Type Casting

Converting one type of data into another type is known as type casting. In C programming language, the data conversion is performed in two different methods as follows...

- **Type Conversion : - Implicit type conversion**
- **Type Casting : - Explicit type conversion**

Type Conversion/ Implicit type conversion

This casting is also known as automatic type casting i.e. type casting done by compiler automatically, when the lower data type is assigned to higher data type. The type of the expression on right hand side and the type of the variable on the left-hand side of an assignment operator may not be same. In such a case, the value of the expression is promoted or demoted depending on the type of the variable on left-hand side of assignment operator (=).

Example: int a;
 float b;
 a=4.6; // **here, there is loss of data**
 b=9;

Example 2: int x=20;
 float y; y=x;
 printf(“%d”=”%f”, x,y); // **here, there is no loss of data**

Explicit type casting:

This casting is also known as **type casting**. Explicit type casting uses **unary type cast operator**. This is useful when higher data type value is trying assigned to lower data type value, because there may be a loss of data. So, the compiler will also show an error like “**possible loss of data**”.

Syntax

(datatype) datavalue or (datatype) variable

Example: float x=20;
 int y,z;
 y=(int)(x+20); // here float value is converted to int type
 z=(int) (x+50);

Example: int a=5,b=2;
 float x;
 x=(float)a/b;